

第七次技术培训

辐射溯源 – 第18届智能车大赛

Technical Support VI

- 工程物理系科协：孙乐然
- 2024 年 4 月 10 日

PART 01

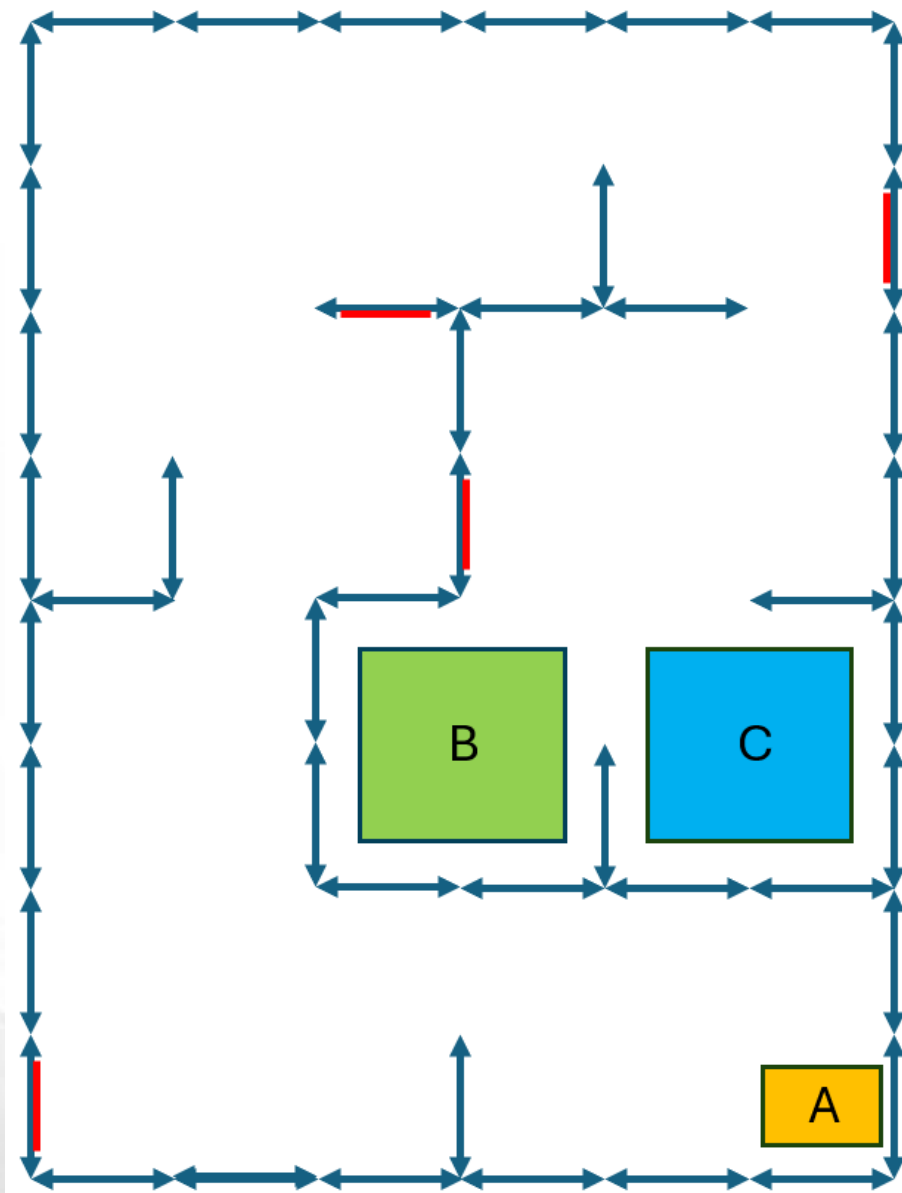
决赛细则

决赛信息

- 时间：4月27日下午
- 地点：科展现场（预计在游泳馆南边的篮球场）
- 不同于初赛使用 radiation 功能包提供辐射强度，决赛使用二维码提供辐射强度。
- 每支队伍有 20 分钟的比赛时间，期间最多进行 5 次尝试，取最好成绩。
- 对于决赛排名，优先比较任务得分，其次比较完成时间。
- 决赛完成时间由**到达辐射源时间、罚时和减时**共同决定。

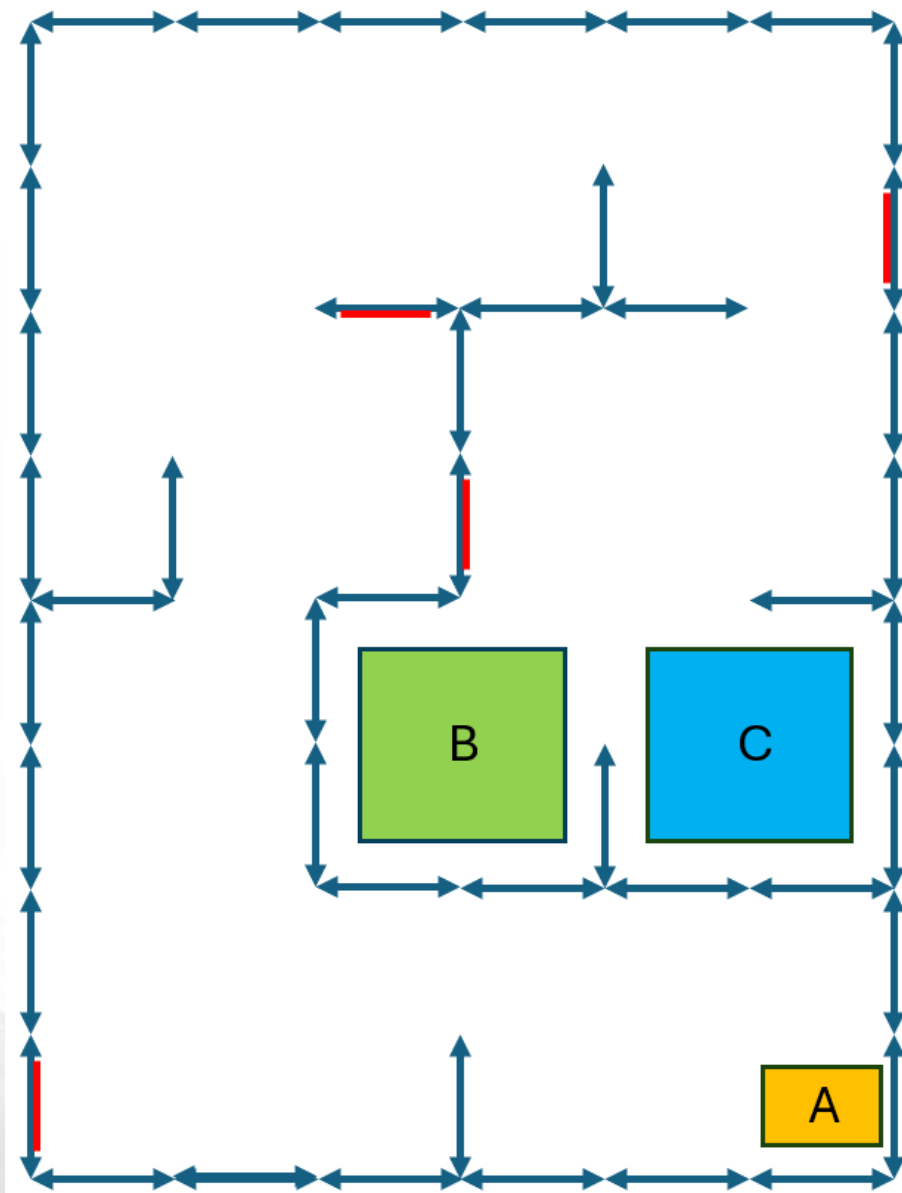
赛道

- 每块板子长0.5m（即图中蓝色箭头）。
- A 为起点区。B、C 为终点区。
- B或C处有一系数未知的辐射源，其辐射强度与距离平方反比。
- 四块板子上贴有二维码（即图中红色短线），其内容为一个数字，表示二维码所在板子中点处的辐射强度。
 - 理论上来说三个数字即可定位辐射源。
 - 选手可以自行选择扫描哪三个二维码，甚至扫描少于三个二维码。
- 整个赛道为 $4\text{m} \times 3\text{m}$ 。



任务列表

- 任务0：决赛前绘制赛道地图。（10分）
- 任务1：使用电脑远程连接智能车，启动程序，智能车经过语音唤醒出发。（10分）
- 任务2：智能车在电脑屏幕输出辐射源位置（误差0.5米内即可）。（20分）
- 任务3：智能车导航至终点区（不一定要正确）。（30分）
- 任务4：智能车根据辐射信号导航至正确终点区。（20分）
- 任务5：到达终点区后语音播放（可自行选择内容）。（10分）



- 除了记录小车完成比赛的用时外，我们还有罚时与减时规则，如下所示：

罚时或减时规则	时长
碰撞墙体或障碍物	用时加3秒
车身与终点区有重合部分， 但未完全进入	用时加4秒
20分钟内未完成比赛	不记录成绩

- 预计4月27日晚上举行答辩会，各组分享自己的体验、心得、创新点。
- 答辩以交流为主，不影响成绩评定。

PART 02

OpenCV

比赛时不一定用得上，但比赛前用得上

OpenCV (Open Source Computer Vision Library) 是一个开源的很强大的图像处理和计算机视觉库，支持C++与Python编程语言，内置多种功能与实现，它提供了很多函数，可以高效地实现各种计算机视觉算法。

OpenCV-Python库

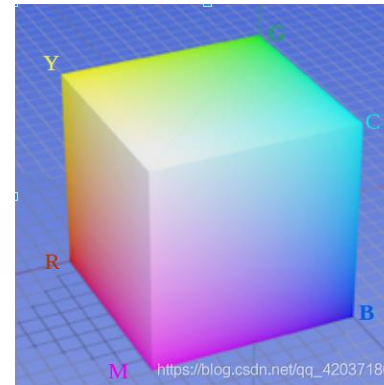
```
import cv2                                # Opencv图像处理库  
from cv_bridge import CvBridge            # ROS与OpenCV图像转换类
```

```
img=cv2.imread(img_path, flags)           #读取图片， 1为彩图， 0为灰度图， -1为原始图  
cv2.imshow(window_name, img)              #显示图片  
cv2.imwrite(img_path, img)                #保存图片  
cv2.waitKey()                             #等待图片关闭  
cv2.destroyAllWindows()                  #销毁所有窗口
```

图像颜色空间

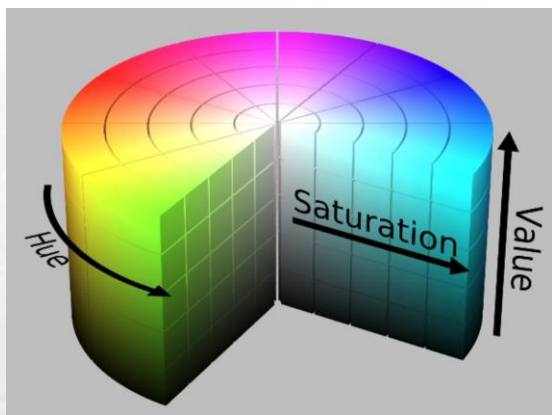
RGB颜色空间:

三个通道表示一幅图像，分别为红色R(Red)，绿色(Green)和蓝色(Blue)。这三种颜色的不同组合可以形成几乎所有的其他颜色。



HSV颜色空间:

三个通道表示一幅图像，分别为色调(Hue)，饱和度(Saturation)和明度(Value)。是针对用户观感的一种颜色模型，侧重于色彩表示，什么颜色、深浅如何、明暗如何。



OpenCV中HSV取值范围:

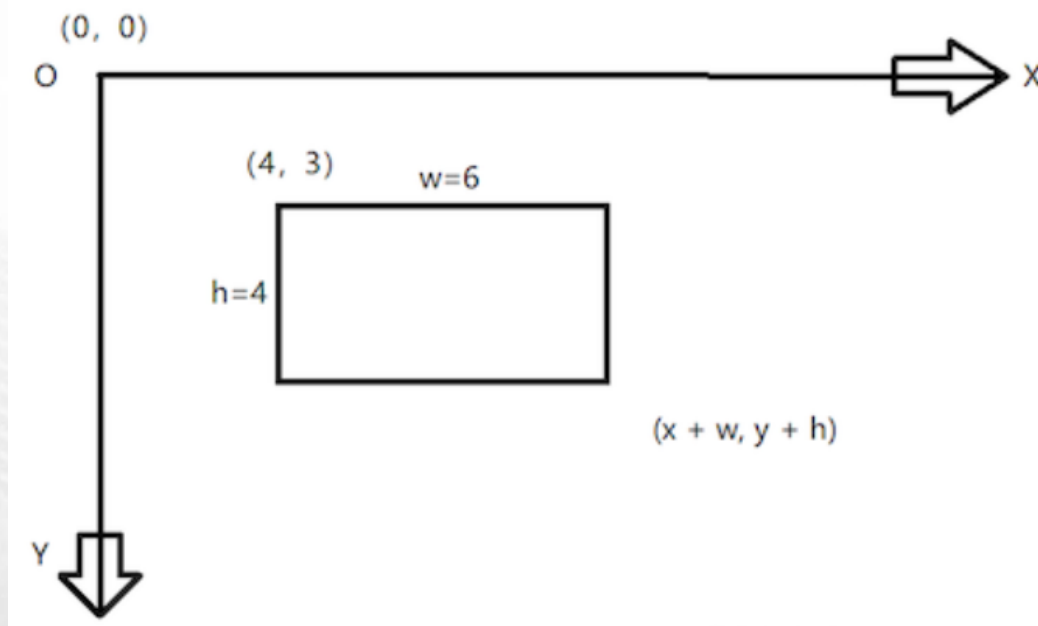
- $H = [0, 179]$
- $S = [0, 255]$
- $V = [0, 255]$

OpenCV常用函数

<code>cv2.cvtColor(image, cv2.COLOR_BGR2HSV)</code>	# 图像从BGR颜色模型转换为HSV模型
<code>cv2.inRange(hsv_img, lower_blue, upper_blue)</code>	# 判断像素值是否在hsv空间的某区间
<code>cv2.findContours()</code>	# 图像中轮廓检测
<code>cv2.drawContours()</code>	# 勾勒轮廓
<code>(x, y, w, h) = cv2.boundingRect(contours)</code>	# 返回轮廓矩形框的边界信息
<code>cv2.bitwise_and(my_image, my_image, mask=mask)</code>	# 图片像素值按位与

(x, y, w, h) 含义:

摄像头所看到的整个视野:
坐标原点是视野左上角, 向
右为x轴正方向, 向下为y轴
正方向:



■ 膨胀

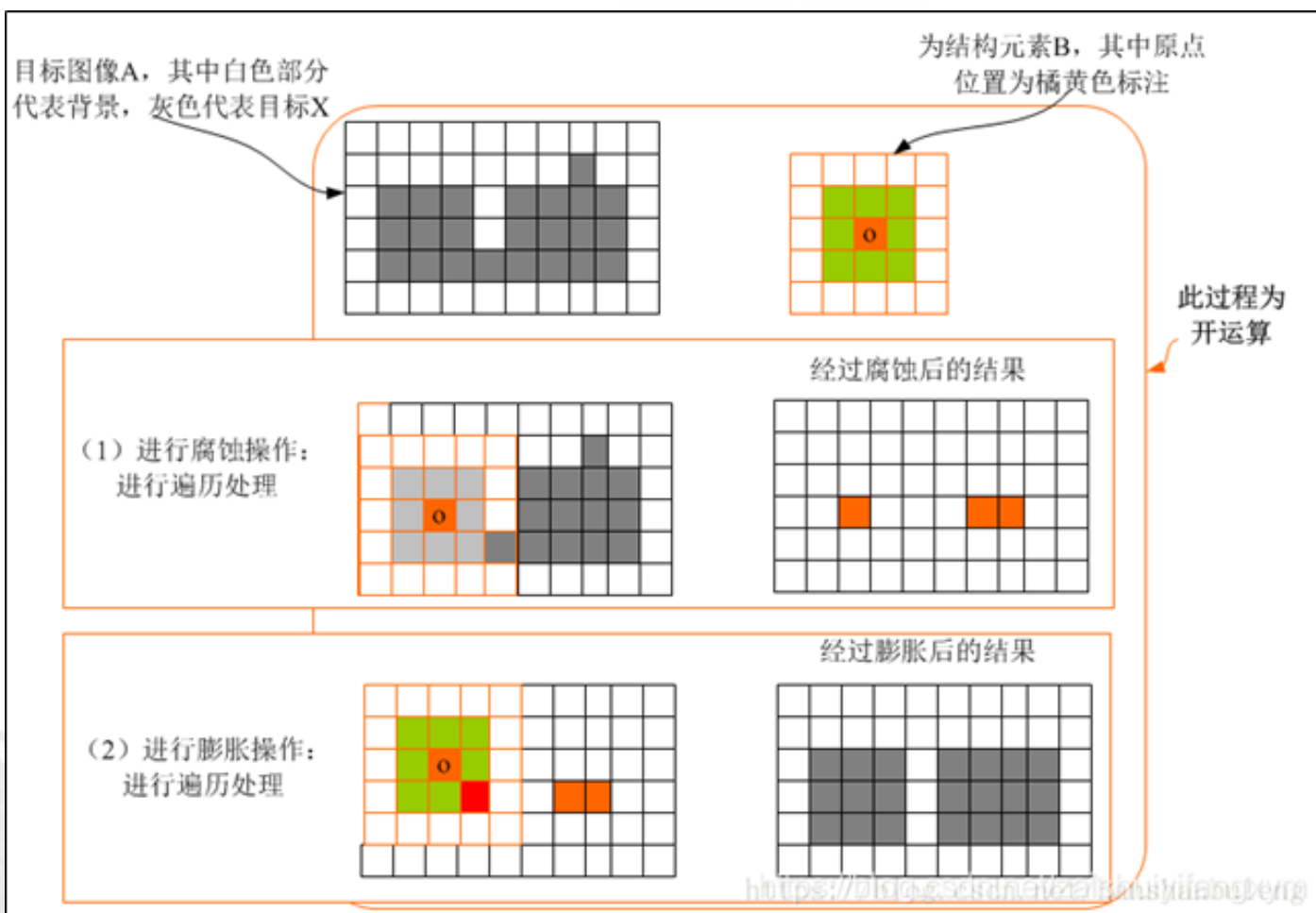
- 使得该像素点的值等于以该像素点为中心的某范围内的最大值。
- `kernel = np.ones((3, 3), dtype=np.uint8)` # 3: 卷积核大小, 也就是对周围 3×3 考虑。
- `dilate = cv2.dilate(img, kernel, 1)` # 1: 迭代次数, 也就是执行几次膨胀操作 (下同)。
- 对于我们的地图, 只有黑、白、灰三色, 膨胀操作使得只要原图片 3×3 范围内有白的, 该像素点就是白的。

■ 腐蚀

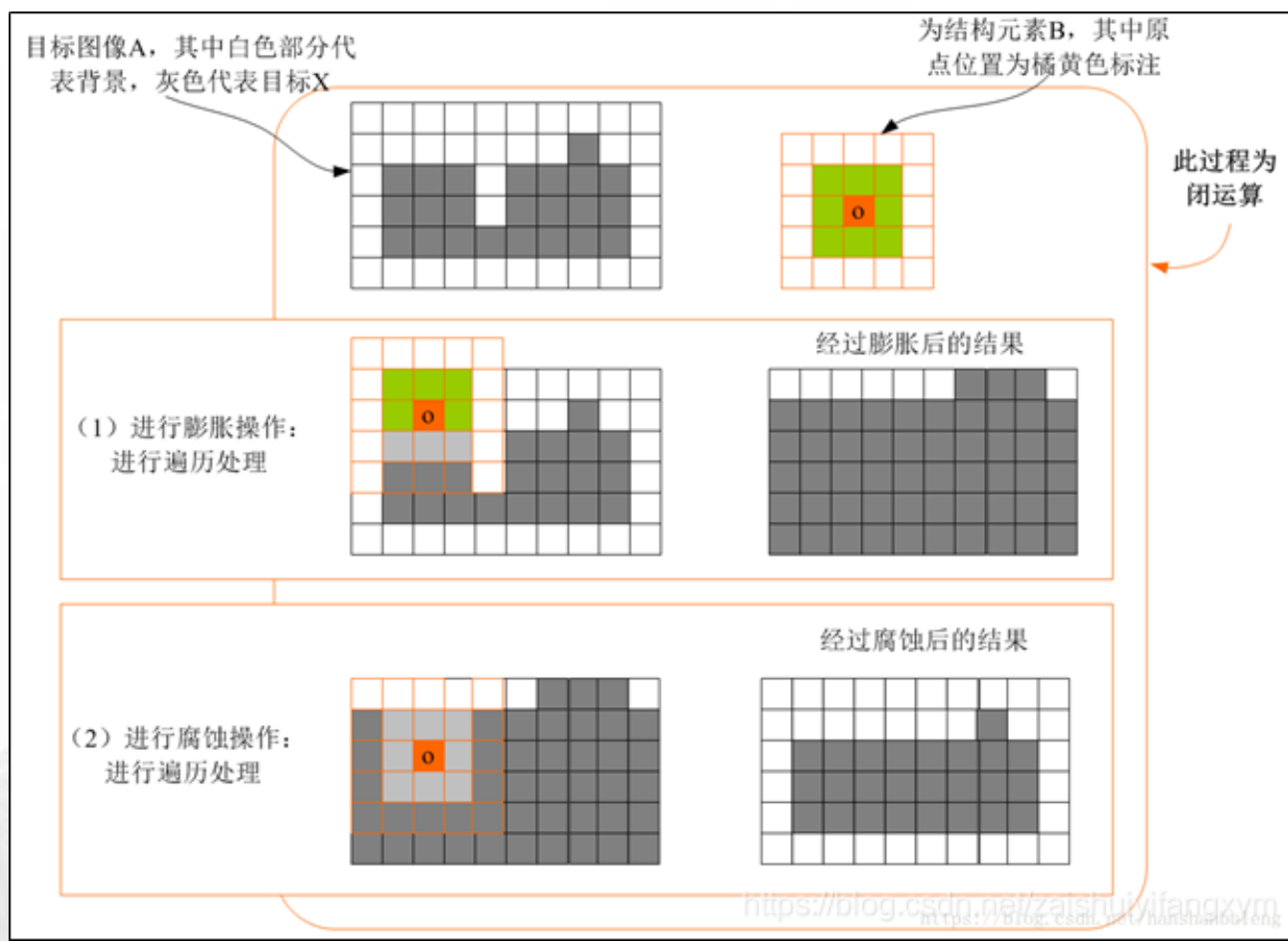
- `erosion = cv2.erode(img, kernel, 1)`
- 与膨胀操作相反, 只要原图片 3×3 范围内有黑的, 该像素点就是黑的。

■ 通过修改卷积核大小和迭代次数, 可以修改膨胀和腐蚀效果。

- 先腐蚀，后膨胀。可以去除噪声，并保留原有图像。
- `opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel, 1)`



- 先膨胀，后腐蚀。填补缺陷，并保留原有图像。
- `closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel, 1)`





PART 03

控制小车运动

- base_driver 功能包是小车的驱动包，我们可以直接使用它控制小车的运动。启动节点：
 - `roslaunch ucar_controller base_driver.launch`
- 发布话题控制：
 - base_driver 节点订阅了 /cmd_vel 话题，msg 类型为 geometry_msgs/Twist。我们可以通过发布 /cmd_vel 话题来控制小车移动。
 - 在终端中使用 rostopic (善用 tab 补全) :

```
ucar@ubuntu:~$ rostopic pub -r 20 /cmd_vel geometry_msgs/Twist "linear:  
  x: 0.0  
  y: 0.0  
  z: 0.0  
angular:  
  x: 0.0  
  y: 0.0  
  z: 1.0"
```

- 输入指令后小车会持续逆时针自转。因为给出的指令是 $\text{angular_z} = 1.0$ 。
- 如果指令 $\text{linear_x} = 1.0$ ，则小车会向前冲出。
- 小车速度小于 0.1 时，会非常缓慢甚至停下，如果调车时遇到这种情况大概率是此问题。
 - 如果使用 dwa 算法，那么解决方法之一是修改 `dwa_local_planner_params.yaml` 里的 `min_vel_x`

```
ucar@ubuntu:~$ rostopic pub -r 20 /cmd_vel geometry_msgs/Twist "linear:  
  x: 0.0  
  y: 0.0  
  z: 0.0  
angular:  
  x: 0.0  
  y: 0.0  
  z: 1.0"
```

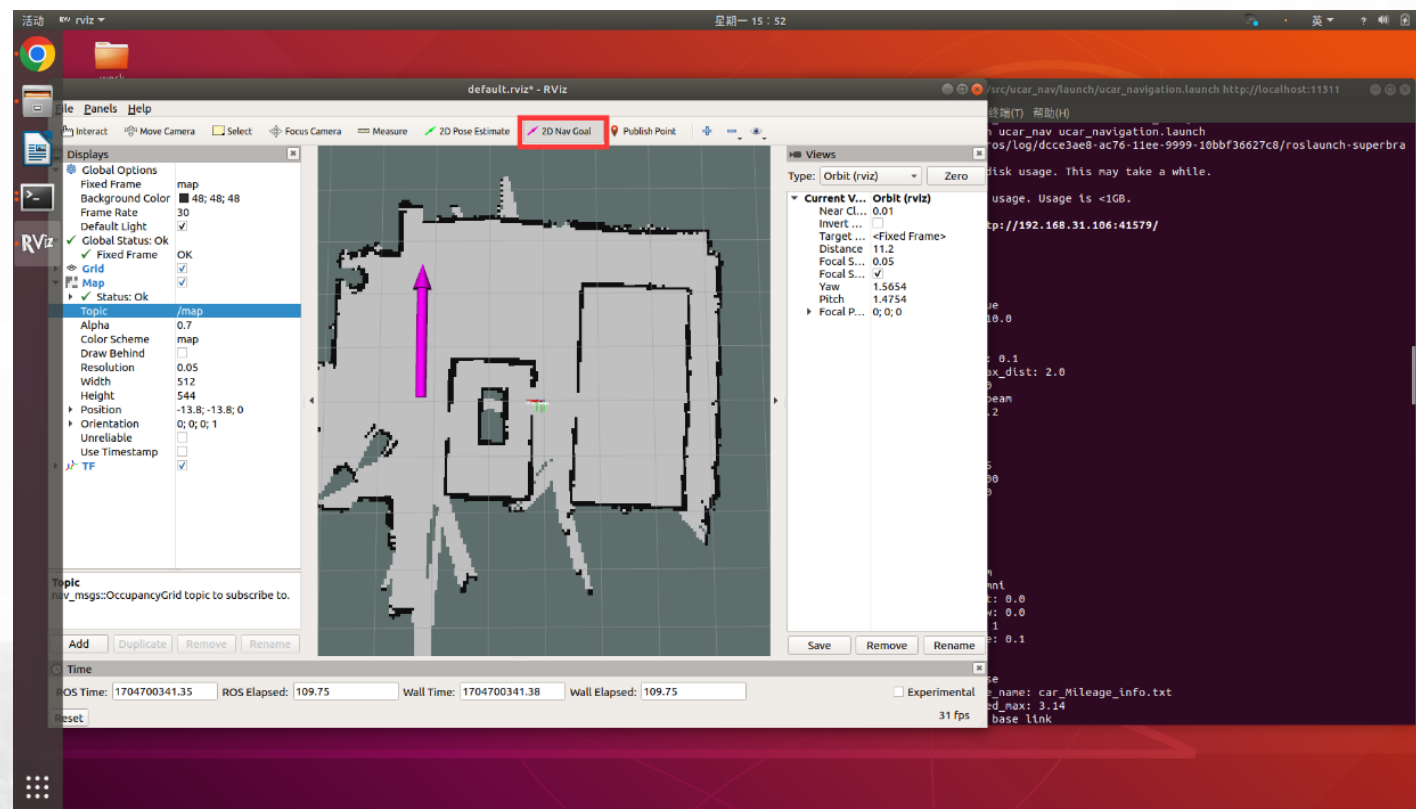
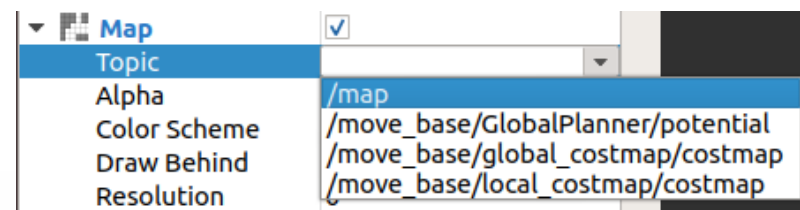
- ucar_nav 包是基于 navigation 的包集，完成了基于 move_base 的导航配置，供参考使用。和初赛使用的 gazebo_nav 对应。
- 要使用它，首先要把建好的地图的 .pgm 和 .yaml 文件放入 /ucar_nav/maps 目录下，并修改 /ucar_nav/launch/ucar_navigation.launch 第 9 行的 map 的文件名为建好的地图的文件名。

```
1 <launch>
2
3   <!-- Run the map server -->
4
5   <include file="$(find ucar_controller)/launch/base_driver.launch" > </include>
6
7   <include file="$(find ydlidar)/launch/ydlidar.launch" > </include>
8
9   <node name="map_server" pkg="map_server" type="map_server" args="$(find ucar_nav)/maps/  
  map_418_002.yaml" output="screen">
```

- ucar_navigation.launch 里包含了驱动包 base_driver 的启动，因此不需要专门启动后者。

ucar_nav

- 在小车终端运行 `roslaunch ucar_nav ucar_navigation.launch`
- 在另一个终端运行 `roslaunch rviz rviz`, 并添加 map 话题,
- 与建图时一样, 需要手动添加map。
- 使用紫色箭头 2D Nav goal 发送终点位置。
 - 理论上这个命令要在电脑终端运行, 但是科协调试的时候电脑端的 rviz 没有成功给小车发送信号。
 - 可以通过键盘和显示器在小车终端发送这个命令, 在小车开始移动的一瞬间拔掉显示器线即可 (乐



- 与初赛同理，我们可以运行 `rostopic echo /move_base_simple/goal`，查看 2D Nav Goal 指向的点位坐标。
- 这个方法可以帮助我们获取一些关键点位坐标（例如二维码位置），写入代码内，控制小车到相应位置。
- 还是与初赛同理，正式比赛时不是用 2D Nav Goal 发送终点，而是在自己的代码中与 `move_base` 建立通信，通过 Action 发送目标。

- 决赛中，选手首先要修改初赛的 launch.sh（当然，有能力的直接写个 .launch 文件也可以），在其中加入 xf_mic_asr_offline（语音输入）、qrcode_tester（二维码识别）、ucar_nav（导航）、决赛主程序等模块的启动。
- 决赛主程序可以在初赛的 simple_navigation_goals 基础上修改：
 - 在开头和结尾加上语音输入和语音输出。
 - 采样辐射信号的点改成扫描二维码的点。
- 也可以使用上节课末尾提到的状态机。

PART 04

调车安排

- 在“预约调车”腾讯文档中填写队伍编号、队伍调车次数。
- 每次调车 3 小时。
 - 提前做完当然更好。
- 尽量本周内完成第一次调车，我会全程陪同。
- 之后的调车：
 - 如果是工作时间，找西主楼 3-224 李老师拿钥匙。
 - 如果是晚上，找我拿钥匙。
 - 详情见云盘的“比赛场地使用指南.pdf”。
- 决赛前每支队伍最好有 3 次左右的调车。

调车准备和调车任务

- 第一次调车前
 - 熟悉第五~第七次技术培训
- 第一次调车
 - 实验各功能模块
 - 初步建图
- 第二次调车前
 - 优化建图
- 第二次调车
 - 完成基本任务
- 第三次调车.....



调车场地示意图（西主楼 3-225）

欢迎交流提问

辐射溯源 – 第18届智能车大赛

Feel free to ask questions!

- 工程物理系科协：孙乐然
- 2024 年 4 月 10 日